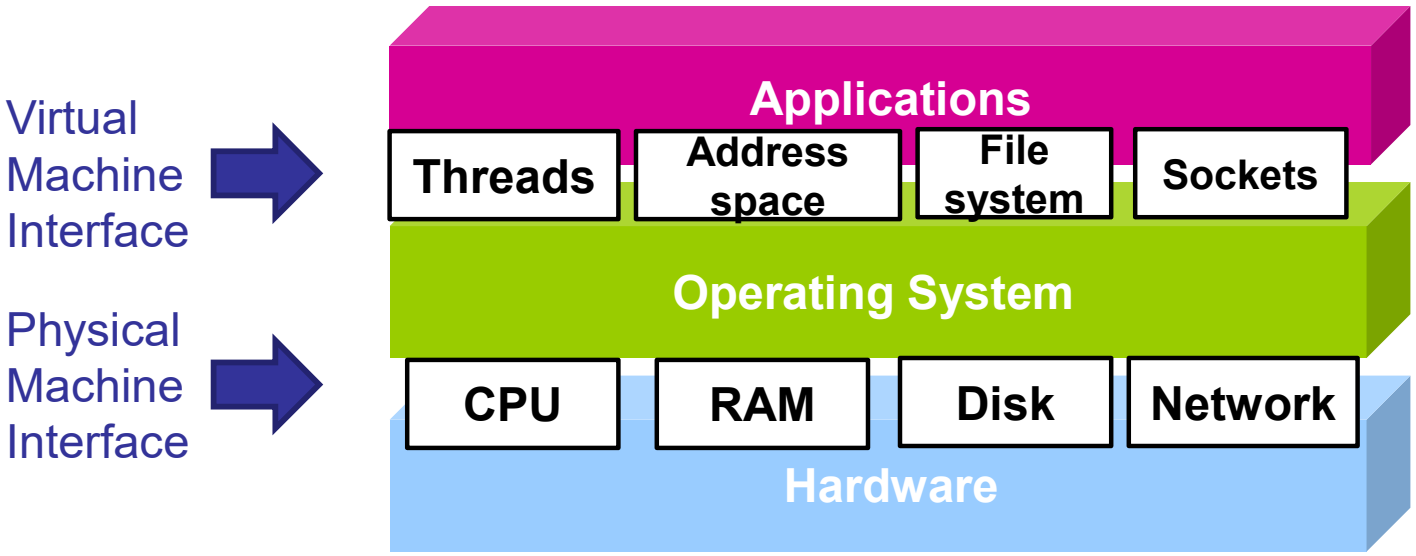
EECS 482
Introduction to operating systems

Networking

Peter Chen
University of Michigan

OS abstractions



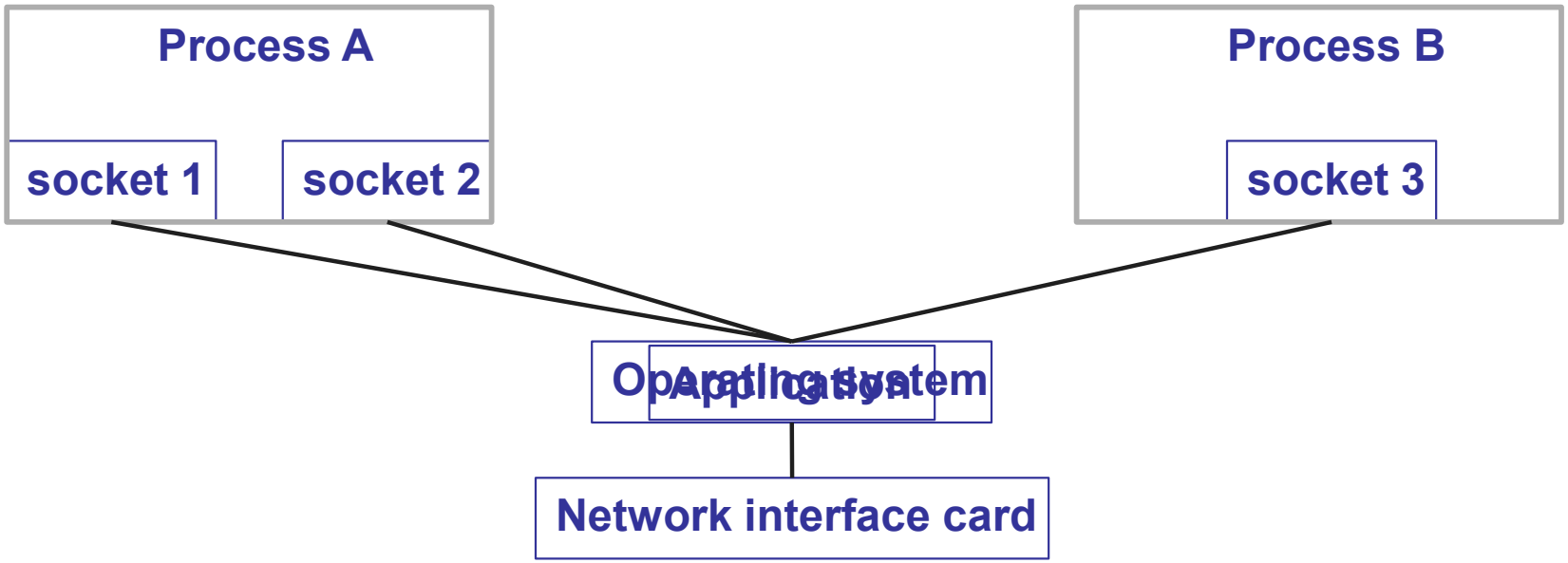
Network abstractions

Hardware reality	**OS abstraction**
Destination named by MAC address of NIC	Destination named by symbolic hostname
Deliver to computer on LAN	Route to final destination across multiple networks
Machine-to-machine	Process-to-process
Unordered, unreliable	Ordered, reliable
Finite-size messages	Byte streams
Multiple distinct computers, each with own resources	Single computer with combined resources

Machine-to-machine communication ➔ Process-to-process communication

- Machine ➔ process
  - Processors ➔ threads
  - DRAM ➔ address spaces
  - Disks ➔ files and directories
  - Network interface card ➔ sockets
- Socket
  - Virtual network interface card
  - Endpoint for communication
  - NIC named by MAC/IP address; socket named by “port number” (via bind)
  - Programming interface: BSD sockets

Socket abstraction



Types of sockets

- UDP (user datagram protocol): IP + sockets
- TCP (transmission control protocol): IP + sockets + reliable, ordered byte streams

- How to provide reliable, ordered byte streams on top of unreliable, unordered packets?

Project 4

- Read manual pages
- Understand C strings
  - What are you passing when you call `fs_writeblock(…, "hello world", …)` ?
- What data should be a C string, and what data need not be a C string?
  - File data?
  - `str` functions assume data is a C string. Caller must guarantee this pre-condition.
- What data is trusted, and what is not trusted?
  - Data sent by client?
  - Initial file system image?

Unordered ➔ ordered messages

- Hardware reality: messages can be re-ordered
  - Sender: msg A, msg B, msg C, msg D, msg E
  - Receiver: msg A, msg B, msg D, msg C, msg E
- OS abstraction: messages received in order sent
  - How to provide this abstraction?
- TCP: connects two sockets with a point-to-point, bidirectional, ordered channel
  - Server calls `accept` to accept incoming connection
  - Client calls `connect` to establish connection with server

Unreliable ➔ reliable messages

- Hardware interface: messages can be
  - Dropped
  - Duplicated
  - Corrupted

- OS abstraction: each message is delivered exactly once (and in the right order)

Unreliable ➔ reliable messages

- How to detect and fix a dropped message?
- How to fix (once you've detected that a message was dropped)?
- How does sender know the message was dropped?
- Extra communication needed
- Is the sender's conclusion always correct?

Unreliable → reliable messages

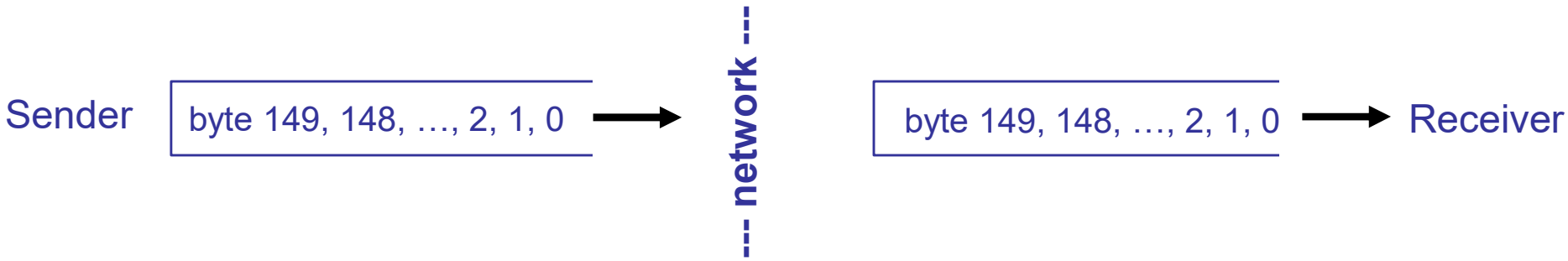
- How to deal with duplicate messages?
- How to deal with corrupted messages?
- Solving problems via transformation
  - Corrupted messages → dropped messages
  - Potential dropped messages → potential duplicates
  - Solve duplicates by adding sequence # and dropping duplicates

Byte streams

- Hardware interface: send/receive distinct messages
- OS abstraction: send/receive infinite stream of bytes
- How to implement the byte stream abstraction?

Using byte streams

- TCP has no message boundaries (unlike UDP)
  - Sender sends bytes; receiver receives bytes



- `n = recv (sock, buf, 150, 0);`
  - Blocks until can receive > 0 bytes (or error)
  - May return [1, 150] bytes
  - To receive > 1 byte, must call recv in loop
    - » or use MSG_WAITALL to receive a known number of bytes

How to send message over byte streams?

- How to know # of bytes to receive?
  - Convention (size specified by protocol)
  - Size specified in header
  - End-of-message delimiter
  - Sender closes connection

- Which of these are used in Project 4?

Client-server

- Common way to structure a distributed application:
  - Server provides some centralized service
  - Client makes request to server, waits for response
- Example: web server
  - Server stores and returns web pages
  - Clients run Web browsers, make GET/POST requests
- Example: coke machine
  - Server manages state associated with coke machine
  - Clients call `client_produce()` or `client_consume()`, which send request to the server and return when done
  - Server responds to client after request has been satisfied

Client-server example

```
client_produce() {
  send produce request to server
  receive response
}

server() {
  while (1) {
    receive request
    if (produce request) {
      add coke to machine
    } else {
      take coke out of machine
    }
    send response
  }
}
```

Problems?

Example with waiting

```
client_produce() {
  send produce request to server
  receive response
}

server() {
  while (1) {
    receive request
    if (produce request) {
      while (machine is full) wait
      add coke to machine
    } else {
      ...
    }
    send response
  }
}
```

Problems?

Example with threads

```
server() {
  while (1) {
    receive request
    if (produce request) {
      create thread that calls server_produce()
    } else {
      create thread that calls server_consume()
    }
  }
}

server_produce() {
  lock
  while (machine is full) wait
  add coke to machine
  signal
  send response
  unlock
}
```

Problems?

Example with threads

```
server() {
  while (1) {
    wait for new connection from client
    create thread that calls handle_request()
  }
}

handle_request() {
  receive request
  call server_produce() or server_consume()
}

server_produce() {
  lock
  while (machine is full) {
    wait
  }
  put coke in machine
  signal
  unlock
  send response
}
```

Alternatives

- How to lower overhead of creating threads?

- How to structure the server to avoid blocking for slow operations
  - Synchronous/blocking: call slow operation and wait for it
    - » Threads
  - Asynchronous/non-blocking: don't call slow operation
    - » Polling (via `select` or `poll`)
    - » Callbacks
    - » What to do if there's no work?
- What are the slow operations in producer/consumer?
- What are the slow operations in Project 4?